

Window Functions for Data Analysis

By Utkarsh Sahu, 29th September 2025

Window functions perform calculations across a set of rows that are related to the current row. Unlike standard aggregate functions (like `SUM()` with `GROUP BY`), which collapse rows into a single output row, window functions return a value for **each and every row**.

The magic of window functions is in the `OVER()` clause. This clause defines the “window” or set of rows the function should operate on. It has two key components:

- **PARTITION BY:** This divides the rows into partitions (groups). The window function is applied independently to each partition and resets when it crosses into a new one. It’s like a temporary `GROUP BY` just for the function.
- **ORDER BY:** This orders the rows within each partition. This is crucial for functions that depend on sequence, like ranking, cumulative sums, or accessing previous/next rows.

Setup: Dummy Sales Table

To demonstrate these functions, we will use a single, consistent `SalesData` table throughout all examples. This table logs individual sales transactions across different product categories.

```
-- Table Initialization
CREATE TABLE SalesData (
    SaleID INT PRIMARY KEY,
    ProductCategory VARCHAR(50),
    SaleDate DATE,
    SaleAmount NUMERIC(10, 2)
);

-- Fill table with some values
INSERT INTO SalesData (SaleID, ProductCategory, SaleDate, SaleAmount) VALUES
(1, 'Electronics', '2025-09-01', 1200.00),
(2, 'Electronics', '2025-09-02', 950.00),
(3, 'Clothing', '2025-09-02', 300.00),
(4, 'Electronics', '2025-09-03', 950.00), -- Tie in SaleAmount for ranking
(5, 'Books', '2025-09-03', 150.00),
(6, 'Clothing', '2025-09-04', 500.00),
(7, 'Books', '2025-09-04', 200.00),
(8, 'Electronics', '2025-09-05', 1500.00),
(9, 'Clothing', '2025-09-05', 300.00); -- Tie in SaleAmount for ranking
```

Ranking Functions

`ROW_NUMBER()`

- **Use Case:** Assigns a unique, sequential number to each row within its partition. It never produces duplicates within a partition.
- **Syntax:** `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`
- **Business Example:** “We need to find the top 2 highest-value sales from each product category to feature in our weekly newsletter.” `ROW_NUMBER()` can uniquely number the sales in descending order of amount, making it easy to filter for the top N.

Query

```
SELECT
    SaleID,
    ProductCategory,
    SaleAmount,
    ROW_NUMBER() OVER(PARTITION BY ProductCategory ORDER BY SaleAmount DESC) AS row_num
FROM
    SalesData;
```

Result

SaleID	ProductCategory	SaleAmount	row_num
7	Books	200.00	1
5	Books	150.00	2
6	Clothing	500.00	1
3	Clothing	300.00	2
9	Clothing	300.00	3
8	Electronics	1500.00	1
1	Electronics	1200.00	2
2	Electronics	950.00	3
4	Electronics	950.00	4

RANK() VS. DENSE_RANK()

RANK() and DENSE_RANK() are similar, but they handle ties differently.

- 
RANK(): Assigns a rank with **gaps** for ties. If two rows tie for 2nd place, they both get rank 2, and the next rank assigned is 4.
- DENSE_RANK():** Assigns a rank with **no gaps** for ties. If two rows tie for 2nd place, they both get rank 2, and the next rank is 3.
- Business Example (RANK()):** "In a sales competition, we need to rank employees. If two people tie for first place, they both get the gold medal, and we skip the silver medal, awarding the next person bronze (3rd place)."
- Business Example (DENSE_RANK()):** "We need to assign products to pricing tiers based on sales. If two products have the same sales, they belong in the same tier, and we don't want to skip any tier numbers."

Query

```

SELECT
    SaleID,
    ProductCategory,
    SaleAmount,
    RANK() OVER(PARTITION BY ProductCategory ORDER BY SaleAmount DESC) AS sales_rank,
    DENSE_RANK() OVER(PARTITION BY ProductCategory ORDER BY SaleAmount DESC) AS sales_dense_rank
FROM
    SalesData;

```

Result

(Focus on the Clothing category where a tie exists for \$300.00)

SaleID	ProductCategory	SaleAmount	sales_rank	sales_dense_rank
...	...	...	...	...

6	Clothing	500.00	1	1
3	Clothing	300.00	2	2
9	Clothing	300.00	2	2
...	...	...	...	...

Notice: For the tied SaleAmount of 300.00, both functions assign rank 2. The next item in a hypothetical list would be rank **4** for RANK() but rank **3** for DENSE_RANK().

Positional Functions

LAG() and LEAD()

- **Use Case:** LAG() accesses data from a previous row, and LEAD() accesses data from a subsequent row within the same partition, without using a self-join.
- **Syntax:** LAG(column, offset, default_value) OVER(...)
- **Business Example (Trend Analysis):** "For each category, we want to calculate the day-over-day change in sales amount. To do this, we need each day's sales figure right next to the previous day's figure." LAG() is perfect for this.

Query

```
SELECT
    ProductCategory,
    SaleDate,
    SaleAmount,
    LAG(SaleAmount, 1, 0) OVER(PARTITION BY ProductCategory ORDER BY SaleDate) AS PreviousDaySale
FROM
    SalesData;
```

Result

ProductCategory	SaleDate	SaleAmount	previousdaysale
Books	2025-09-03	150.00	0.00
Books	2025-09-04	200.00	150.00
Clothing	2025-09-02	300.00	0.00
Clothing	2025-09-04	500.00	300.00
Clothing	2025-09-05	300.00	500.00
Electronics	2025-09-01	1200.00	0.00
Electronics	2025-09-02	950.00	1200.00
Electronics	2025-09-03	950.00	950.00
Electronics	2025-09-05	1500.00	950.00

Bucketing and Aggregation Functions

NTILE(n)



- **Use Case:** Divides the rows within a partition into a specified number of n roughly equal groups or “buckets”.
- **Syntax:** NTILE(n) OVER (PARTITION BY ... ORDER BY ...)
- **Business Example (Customer Segmentation):** “We want to segment our sales into performance quartiles (4 groups). The top 25% of sales will be labeled ‘Premium’, the next 25% ‘Standard’, etc. This helps us analyze the characteristics of high-value transactions.”

Query

```
SELECT
    SaleID,
    ProductCategory,
    SaleAmount,
    NTILE(4) OVER(PARTITION BY ProductCategory ORDER BY SaleAmount DESC) as sales_quartile
FROM
    SalesData;
```

Result

(Showing the ‘Electronics’ category which has 4 rows, fitting perfectly into 4 quartiles)

SaleID	ProductCategory	SaleAmount	sales_quartile
8	Electronics	1500.00	1
1	Electronics	1200.00	2
2	Electronics	950.00	3
4	Electronics	950.00	4



SUM() OVER() and AVG() OVER()

- **Use Case:** Calculates a cumulative (running) aggregate or a moving aggregate over a window of rows.
- **Business Example (Cumulative Growth & Moving Averages):**
 - **SUM():** “We need a report that shows the running total of sales revenue for each category throughout the month to track our progress towards our monthly goal.”
 - **AVG():** “To smooth out daily volatility, we want to calculate a 3-day moving average for sales in each category. This will give us a clearer view of the underlying sales trend.”

Query

```
SELECT
    ProductCategory,
    SaleDate,
    SaleAmount,
    -- Cumulative sum for the category, ordered by date
    SUM(SaleAmount) OVER(PARTITION BY ProductCategory ORDER BY SaleDate) AS RunningTotal,
    -- 3-day moving average for the category, ordered by date
    AVG(SaleAmount) OVER(
        PARTITION BY ProductCategory
        ORDER BY SaleDate
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS MovingAverage3Day
FROM
    SalesData;
```

Result (for 'Electronics')

ProductCategory	SaleDate	SaleAmount	runningtotal	movingaverage3day
Electronics	2025-09-01	1200.00	1200.00	1200.00
Electronics	2025-09-02	950.00	2150.00	1075.00
Electronics	2025-09-03	950.00	3100.00	1033.33
Electronics	2025-09-05	1500.00	4600.00	1133.33

Explanation:

- **RunningTotal:** On Sep 2, the total is $1200 + 950 = 2150$. On Sep 3, it's $2150 + 950 = 3100$.
- **MovingAverage3Day:** On Sep 3, the average is of that day (950), the previous day (950), and the day before that (1200). $(1200+950+950)/3 = 1033.33$.